



SierraCol EPP — Sistema de Visión por Computadora para Seguridad Industrial

Monitoreo inteligente de EPP, evaluación ergonómica REBA/MAC y detección de riesgos en tiempo real para zonas de perforación en Sierracol Energy.



Descripción

SierraCol EPP es un sistema de visión por computadora de grado industrial, diseñado para operar en **tiempo real** sobre flujos de video RTSP en zonas de perforación de Sierracol Energy. Su propósito es detectar y evaluar automáticamente tres dimensiones críticas de la seguridad laboral:

1. **Detección de EPP (Equipos de Protección Personal):** Identifica si los trabajadores dentro de una zona de trabajo definida portan casco, guantes y botas de seguridad, generando alertas ante incumplimientos confirmados.
2. **Evaluación Ergonómica REBA:** Calcula en tiempo real el score [REBA \(Rapid Entire Body Assessment\)](#) completo (Score A + Score B + Score Total) utilizando la pose corporal estimada de cada persona, detectando posturas de alto riesgo ergonómico.
3. **Evaluación de Manipulación Manual de Cargas (MAC):** Implementa el protocolo [MAC \(Manual Handling Assessment Charts\)](#) para detectar escenarios de levantamiento de carga y evaluar el riesgo biomecánico asociado.

El sistema opera de forma autónoma 24/7 dentro de contenedores Docker, registra todos los eventos en una base de datos SQLite local, graba clips de video de los incidentes y, mediante un scheduler programado, consolida, transforma y sube los datos a **Azure Blob Storage** para su consumo desde dashboards de BI.

Arquitectura del Proyecto

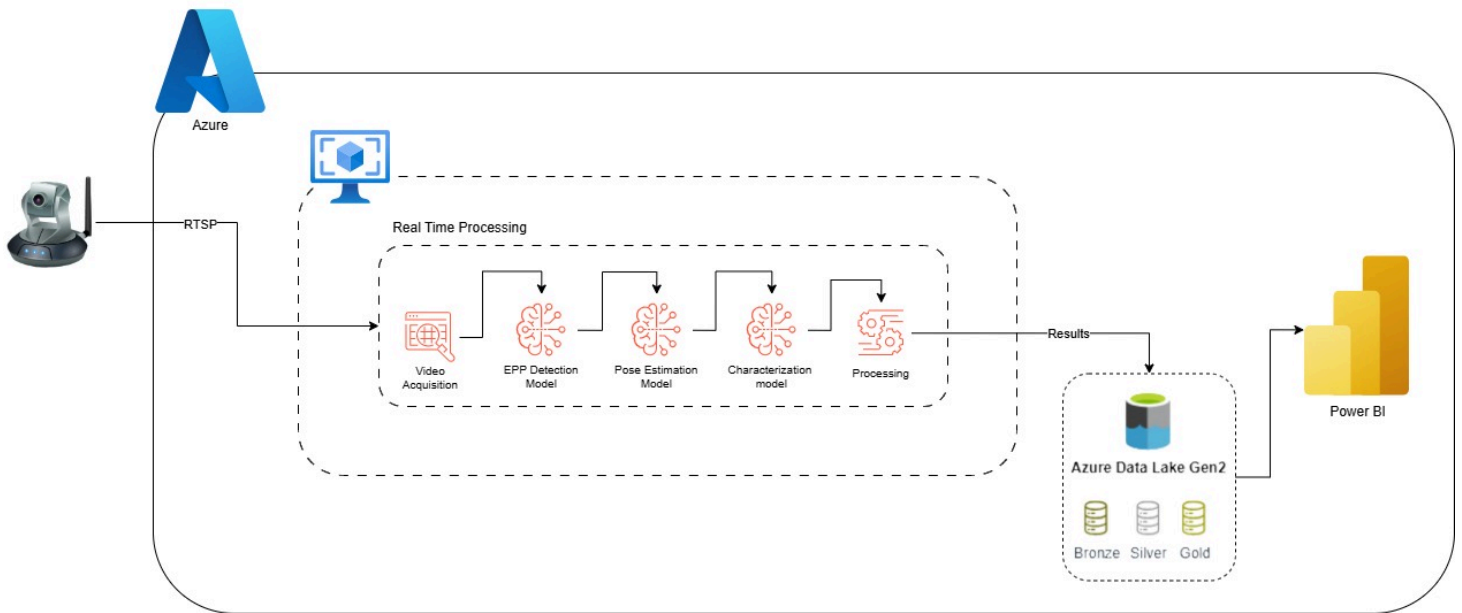
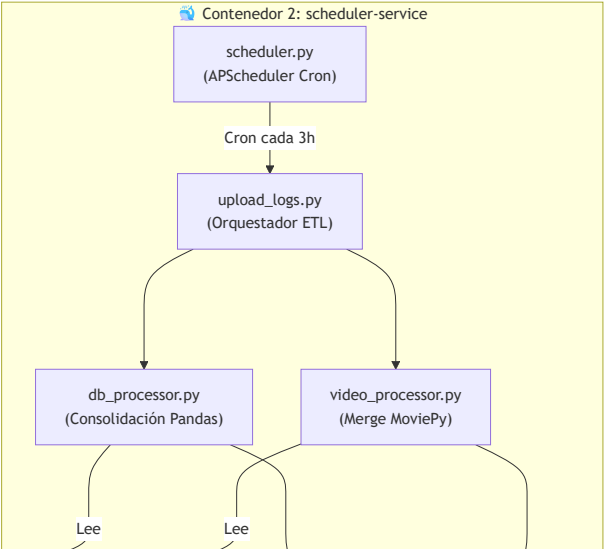
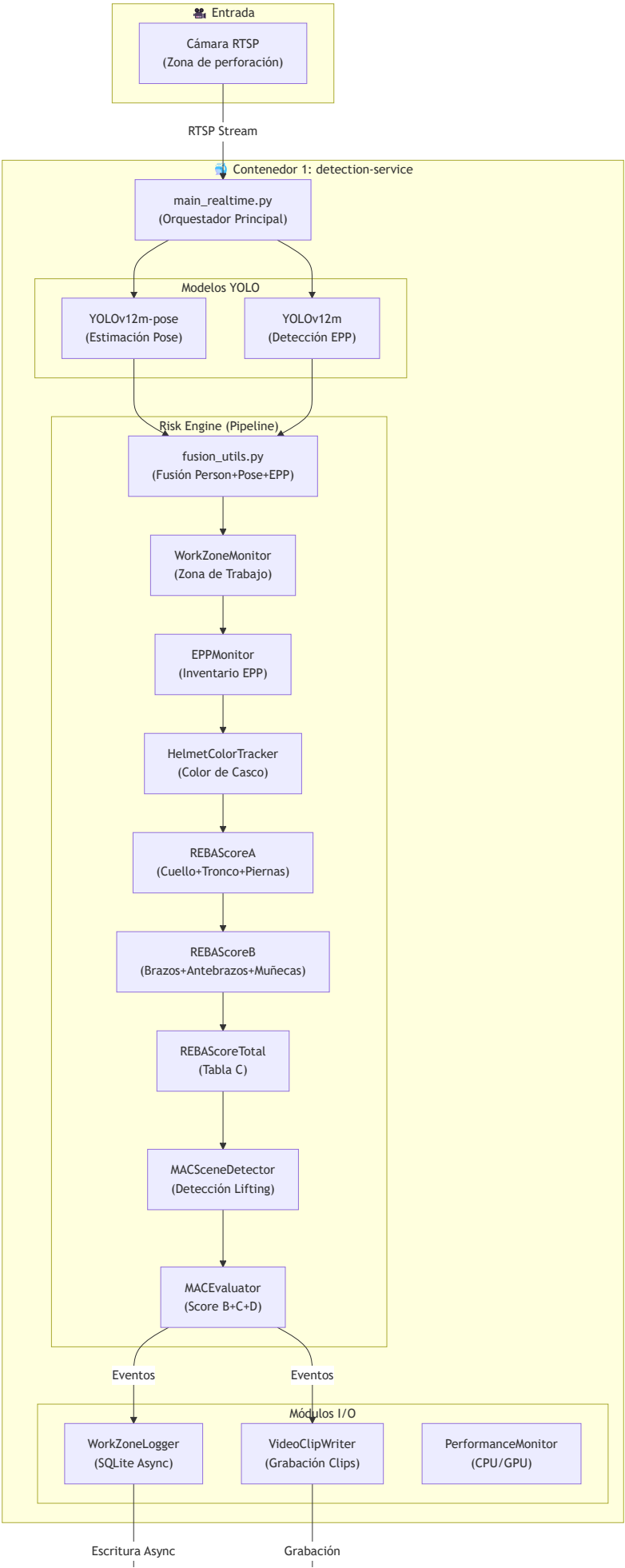


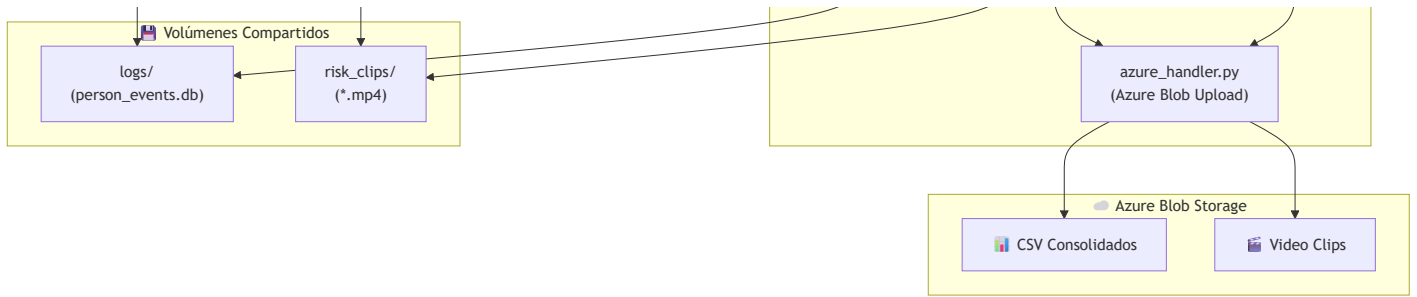
Tabla de Contenidos

-  Descripción
-  [Arquitectura del Proyecto](#)
-  [Arquitectura y Flujo del Sistema](#)
-  [Características Principales](#)
-  [Stack Tecnológico](#)
-  [Estructura del Proyecto](#)
-  [Guía de Instalación y Configuración](#)
-  [Uso y Ejemplos de Ejecución](#)
-  [Variables de Entorno](#)

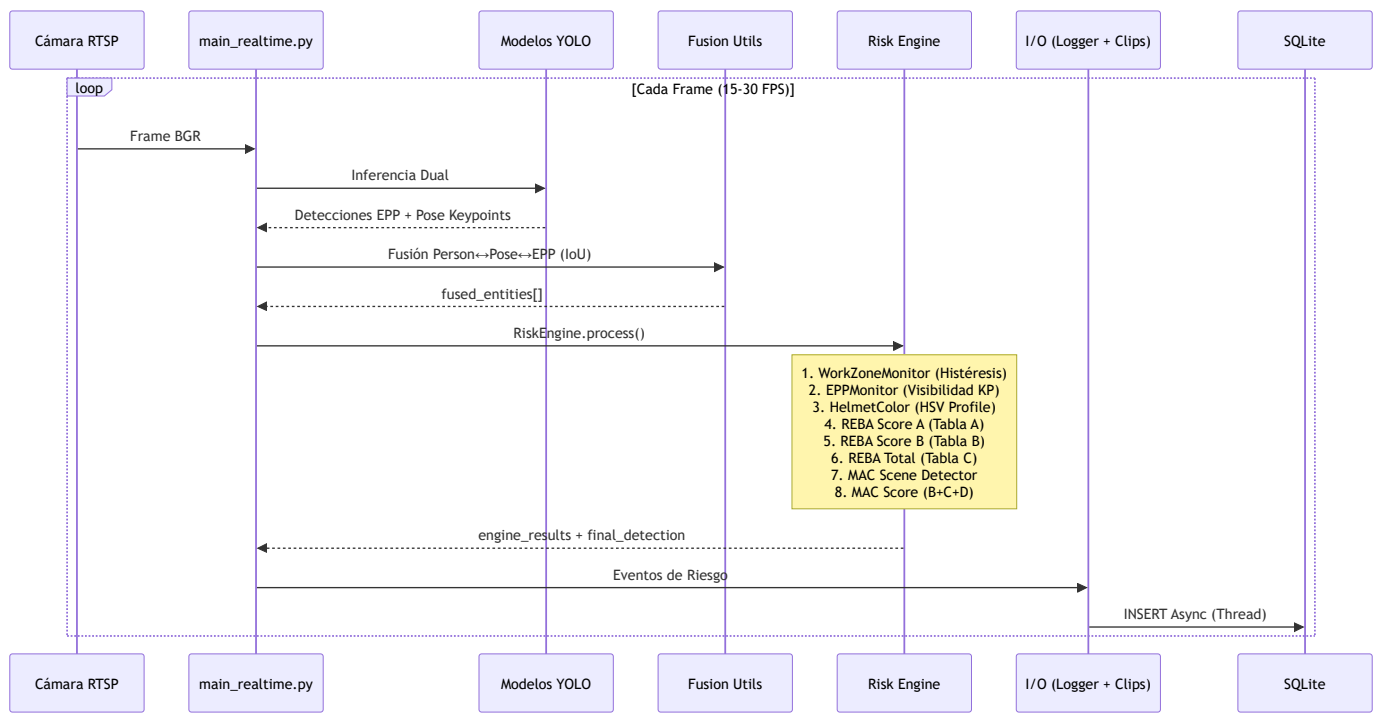
Arquitectura y Flujo del Sistema

El sistema opera con una arquitectura de **dos microservicios** Docker independientes que comparten volúmenes de datos:





Flujo de Procesamiento por Frame



★ Características Principales

Característica	Descripción	Módulo Responsable
Detección de EPP	Detecta casco, guantes, botas y gafas de seguridad con histéresis temporal para evitar falsos positivos. Valida visibilidad de extremidades antes de evaluar.	engine/epp_monitor.py
Evaluación REBA Score A	Calcula score de cuello, tronco y piernas (Neck + Trunk + Legs → Tabla	engine/reba_score_a.py

Característica	Descripción	Módulo Responsable
	A + Load) con validación de orientación lateral.	
Evaluación REBA Score B	Calcula score de brazos superiores, antebrazos y muñecas (Upper Arm + Lower Arm + Wrist → Tabla B).	engine/reba_score_b.py
REBA Score Total	Combina Score A y Score B mediante Tabla C para obtener el nivel de riesgo ergonómico final (1-12).	engine/reba_score_total.py
MAC Scene Detector	Detecta escenarios de levantamiento de carga verificando 4 condiciones simultáneas: pie en zona, pose válida, flexión de rodilla y ángulo de tronco.	engine/mac_scene_detector.py
MAC Score Calculator	Calcula componentes B (distancia manos-espalda), C (zona vertical de levantamiento) y D (torsión/flexión lateral del tronco).	engine/mac_score.py
Monitoreo de Zona de Trabajo	Define un polígono configurable y confirma presencia con histéresis bidireccional (entrada/salida).	engine/work_zone_monitor.py
Clasificación de Color de Casco	Identifica color del casco (blanco, amarillo, naranja) con perfiles HSV adaptativos día/noche.	engine/helmet_color.py
Grabación de Clips	Grabación asíncrona multi-escena con buffer de pre-roll y duración configurable por tipo de evento.	in_out/video_clip_writer.py
Logging Asíncrono SQLite	Escritura no-bloqueante en hilo separado con tabla normalizada para todos los eventos (REBA, MAC, EPP, Zona).	in_out/work_zone_logger.py
ETL Programado	Scheduler Cron que consolida SQLite → Pandas → CSV, merge de video	upload_data/upload_logs.py

Característica	Descripción	Módulo Responsable
	clips y subida a Azure Blob Storage con URLs SAS.	
Proyección de Keypoints Virtuales	Extiende el esqueleto COCO con puntos proyectados de pies y manos para evaluación EPP más precisa.	utils/keypoints_projections.py
Fusión Optimizada	Matching único Person↔Pose↔EPP usando IoU y containment ratio, eliminando cálculos duplicados.	utils/fusion_utils.py

Stack Tecnológico

Categoría	Tecnología	Propósito
Lenguaje	Python 3.10	Lenguaje principal del proyecto
Deep Learning	PyTorch 2.3 + CUDA 12.1	Framework de inferencia con aceleración GPU
Detección de Objetos	Ultralytics YOLOv26	Modelos de detección EPP y estimación de pose
Tracking	Supervision (ByteTrack)	Seguimiento multi-objeto por ID
Visión por Computadora	OpenCV 4.x	Captura RTSP, procesamiento de imagen, grabación
Geometría	Shapely + NumPy	Cálculo de IoU, containment, polígonos de zona
Base de Datos	SQLite 3	Almacenamiento local de eventos con escritura async
Procesamiento de Datos	Pandas	Consolidación y transformación ETL

Categoría	Tecnología	Propósito
Procesamiento de Video	MoviePy + FFmpeg	Concatenación de clips de video de incidentes
Almacenamiento Cloud	Azure Blob Storage	Persistencia de CSV y videos con URLs SAS
Scheduler	APScheduler	Programación de tareas ETL tipo Cron
Contenedorización	Docker + Docker Compose	Despliegue en contenedores con soporte GPU (NVIDIA)
Monitoreo	psutil + pynvml	Métricas de rendimiento CPU/GPU

Estructura del Proyecto

sierracol-epp-27323/

```
|— 📄 docker-compose.yml      # Orquestación de los 2 servicios Docker
|— 📄 .env.example            # Plantilla de variables de entorno
|— 📄 .gitignore              # Reglas de exclusión del repositorio
|— 📄 CHANGELOG.md           # Historial de cambios del proyecto
|— 📄 extract_frame.py        # Utilidad para capturar un frame RTSP
|
|— 📁 risk_detection/         # — SERVICIO 1: Detección de Riesgos —
|   |— 📄 Dockerfile          # Imagen Docker (pytorch/pytorch:2.3-cuda12.1)
|   |— 📄 requirements.txt     # Dependencias del servicio
|   |— 📄 config.py           # Configuración centralizada (umbrales, tablas REBA/MAC)
|   |— 📄 main_realtime.py     # Punto de entrada – Bucle principal de procesamiento
|   |— 📄 risk_engine.py       # Orquestador del pipeline de evaluación de riesgos
|   |
|   |— 📁 engine/              # Motores de evaluación
|       |— 📄 base_scene.py    # Clase abstracta base para todos los monitores
|       |— 📄 models.py        # Dataclasses: Person y Detection
|       |— 📄 work_zone_monitor.py # Detección de presencia en zona (polígono + histéresis)
|       |— 📄 epp_monitor.py   # Inventario de EPP con validación de visibilidad
|       |— 📄 helmet_color.py  # Clasificación de color de casco (HSV día/noche)
|       |— 📄 reba_score_a.py  # REBA Score A: cuello, tronco, piernas (Tabla A)
|       |— 📄 reba_score_b.py  # REBA Score B: brazos, antebrazos, muñecas (Tabla B)
|       |— 📄 reba_score_total.py # REBA Total: Score A × Score B → Tabla C
|       |— 📄 mac_scene_detector.py # Detección de escenario de levantamiento de carga
|       |— 📄 mac_score.py     # MAC Score: componentes B, C y D
|       |
|       |— 📁 in_out/          # Módulos de entrada/salida
|           |— 📄 work_zone_logger.py # Logger asíncrono de eventos → SQLite
|           |— 📄 video_clip_writer.py # Grabador asíncrono de clips multi-escena
|           |— 📄 performance.py   # Monitor de rendimiento (CPU/GPU/FPS)
|           |— 📄 helmet_color_logger.py # Logger específico de color de casco
|           |
|           |— 📁 utils/        # Utilidades de cálculo
|               |— 📄 fusion_utils.py # Fusión Person↔Pose↔EPP (IoU + containment)
|               |— 📄 body_angles.py  # Cálculo de ángulos corporales (tronco, rodilla, cuello)
|               |— 📄 geometry_utils.py # Funciones geométricas (IoU, ángulos entre vectores)
|               |— 📄 keypoints_projections.py # Proyección de keypoints virtuales (pies, manos)
|               |— 📄 pose_utils.py   # Normalización de pose
|               |— 📄 visualization.py # Renderizado de HUD, esqueleto y overlays
|               |
|               |— 📄 visualization.py # Renderizado de HUD, esqueleto y overlays
```



```
cp .env.example .env
```

Important

Revisar la sección [Variables de Entorno](#) para una descripción detallada de cada variable.

3. Colocar los Modelos Entrenados

Asegúrate de que los pesos de los modelos estén en la carpeta correcta:

```
risk_detection/trained_model/  
├─ yolo26m-yoloep-sintest-aumentodatax3.pt  # Modelo de detección de EPP  
└─ yolo26m-pose.pt                        # Modelo de estimación de pose
```

4. Despliegue con Docker Compose (Producción)

```
# Construir y levantar ambos servicios en segundo plano  
docker compose up --build -d
```

```
# Verificar estado de los contenedores  
docker compose ps
```

```
# Ver logs en tiempo real del servicio de detección  
docker compose logs -f detection-service
```

```
# Ver logs del scheduler de carga  
docker compose logs -f scheduler-service
```

```
# Detener todos los servicios  
docker compose down
```

Note

El servicio `detection-service` usa `network_mode: "host"` para acceder a cámaras en la red local y `runtime: nvidia` para aceleración GPU. El servicio `scheduler-service` es un contenedor ligero basado en `python:3.10-slim`.

5. Instalación Local (Desarrollo)

Si prefieres ejecutar sin Docker para desarrollo o depuración:

```
# Crear entorno virtual
python -m venv .venv

# Activar entorno (Windows)
.\.venv\Scripts\Activate

# Instalar dependencias del servicio de detección
pip install -r risk_detection/requirements.txt

# Instalar dependencias del servicio de carga (opcional)
pip install -r upload_data/requirements.txt
```

Uso y Ejemplos de Ejecución

Ejecución del Sistema de Detección (Tiempo Real)

```
# Desde el directorio risk_detection/
cd risk_detection
python main_realtime.py
```

Salida esperada:

```
2026-06-12 08:00:01 | INFO | Iniciando Sistema de Vision (EPP + Pose + Zonas)...
2026-06-12 08:00:03 | INFO | Modelos cargados en 2.15s
2026-06-12 08:00:03 | INFO | Logs de WorkZone (event-based) -> logs/person_events.db
2026-06-12 08:00:03 | INFO | 📹 VideoClipWriter inicializado. Clips se guardarán en: risk_clips
2026-06-12 08:00:04 | INFO | Conectando a video fuente: rtsp://...
2026-06-12 08:00:04 | INFO | FPS: 15.00 | Buffer pre-roll: 30 frames
2026-06-12 08:00:04 | INFO | Sistema completamente inicializado.
```

Con `VISUALIZE=True` , se abrirá una ventana OpenCV mostrando el frame anotado con:

- Polígono de zona de trabajo (verde/naranja)
- Bounding boxes de personas y EPP con labels

- Esqueleto de pose superpuesto
- Scores REBA y MAC en el HUD

Presionar `q` para detener la ejecución.

Ejecución del Scheduler de Carga

```
# Desde el directorio upload_data/  
cd upload_data  
python scheduler.py
```

Salida esperada:

```
2026-06-12 19:00:00 | INFO | Iniciando servicio de programación (scheduler-service)...  
2026-06-12 19:00:00 | INFO | ● [Scheduler] Servicio de programación iniciado.  
2026-06-12 19:00:00 | INFO | Zona horaria: America/Bogota  
2026-06-12 19:00:00 | INFO | Tarea programada para las horas: 1,4,7,10,13,16,19,22 en el mir
```

Captura de un Frame de Prueba (Utilidad)

Para verificar la conectividad con la cámara RTSP:

```
python extract_frame.py
```

Se guardará un frame como `camara_frame.png` en el directorio raíz.

Variables de Entorno

El archivo `.env` controla toda la configuración del sistema. Copiar desde `.env.example` :

Detección de Eventos

Variable	Descripción	Ejemplo
RTSP_URL	URL completa del stream RTSP de la cámara	rtsp://user:pass@192.168.1.10:554/ch1

Variable	Descripción	Ejemplo
VISUALIZE	Habilitar ventana de visualización en tiempo real	True / False
MONITOR_PERFORMANCE	Activar tracking de métricas CPU/GPU	True / False

Grabación de Clips

Variable	Descripción	Ejemplo
CLIP_ENABLED	Activar grabación de clips de eventos	True
CLIP_PREROLL_SEC	Segundos de video ANTES del evento detectado	2
CLIPS_DIR	Directorio de salida de clips (mapeado por Docker)	risk_clips

Registros y Logs

Variable	Descripción	Ejemplo
LOG_DIR	Directorio de archivos .db y consolidados	logs

Scheduler y Carga

Variable	Descripción	Ejemplo
TIMEZONE	Zona horaria para el scheduler	America/Bogota
GAP_THRESHOLD_SECONDS	Segundos máximos entre frames con eventos para unificarlos	10
HOURS_SCHEDULER_ACTIVE	Horas de activación del cargue al storage	[1, 4, 7, 10, 13, 16, 19, 22]
MINUTE_SCHEDULER_ACTIVE	Minuto de activación dentro de cada hora	2

Azure Storage

Variable	Descripción	Ejemplo
AZURE_ACCOUNT_NAME	Nombre de la cuenta de Azure Storage	miStorageAccount
AZURE_STORAGE_ACCOUNT_URL	Endpoint del Storage Account	https://mi.blob.core.windows.net
AZURE_STORAGE_CONNECTION_STRING	Cadena de conexión completa	DefaultEndpointsProtocol=https;...
AZURE_CONTAINER_NAME	Nombre del contenedor Blob	epp-data
AZURE_CSV_PATH	Carpeta destino para archivos CSV	events_detection_csv_results
AZURE_VIDEO_PATH	Carpeta destino para clips de video	events_detection_video_clips_results